



Universidad autónoma de baja California

Ingeniería en computación

Sistemas operativos

Practica N.6 procesos

Alma Leticia Palacios Guerreo

Garcia Chávez Erik

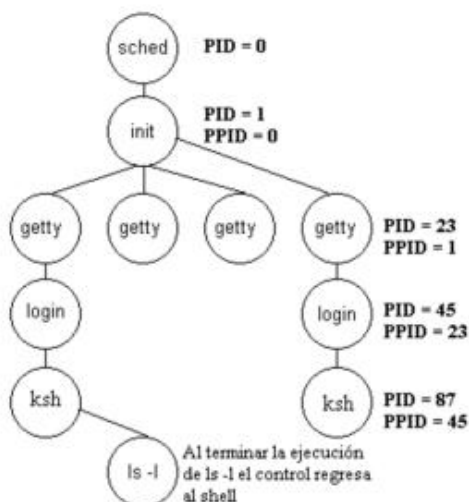
martes 28 de octubre del 2025

Procesos:

Es una entidad dinámica que consiste en un programa en ejecución, sus valores actuales, su estado y los recursos utilizados para manejar su ejecución estos son: memoria, CPU, dispositivos de E/S, etc. Pueden coexistir varias instancias de un mismo programa en ejecución en forma simultánea. ya que cada una de ellas es un proceso diferente.

Programa: Un programa es una entidad inactiva, estática, la cual consiste de un conjunto de instrucciones y datos asociados. Si un programa es invocado varias veces puede generar múltiples procesos.

Los sistemas unix son un sistema de multiprocesos por tiempo compartido, aunque parezca que muchos procesos se están ejecutando concurrentemente desde el punto de vista del proceso, este tiene el acceso y control de todos los recursos del sistema como si fuera único. los sistemas Unix corren en plataformas que tienen una unidad de procesamiento capaz de soportar muchos procesos activos; sin embargo, en un instante dado de tiempo solamente un proceso puede estar en ejecución. aparentan que trabajan en varios procesos al mismo tiempo, pero en realidad existe una distribución temporal de la asignación de CPU entre los diferentes procesos que compiten por ejecutarse. La conmutación temporal de procesos, está basada en un sistema de colas FIFO multinivel con actualización de prioridad. Los sistemas con múltiples unidades de procesamiento, las cuales por definición pueden soportar un verdadero procesamiento concurrente se dice que tienen capacidad de multiprocesamiento. Podemos ver un poco de la jerarquía que tienen los procesos:



Tipo de procesos:

- **Procesos del sistema:** Son los procesos que actúan sin que el usuario los solicite, también se les conoce como dejamos. Pueden ser de dos tipos:
 - 1- Procesos permanentes o de larga duración: Se crean al arrancar el sistema y permanecen activos hasta que se termina la conexión. Su función es realizar las actividades del sistema.
 - 2- Procesos transitorios: Nacen y mueren cuando el sistema efectúa tareas propias, independientes de los usuarios.
- **Procesos de usuario:** Son los procesos creados cuando el usuario ejecuta comandos.
- **Atributos del proceso:**
 - Cada proceso tiene un identificador único (PID). Este es un entero no negativo asignado por el sistema. Garantiza que el proceso sea único dentro del sistema
 - Un proceso tiene asignados tres segmentos de memoria: Segmento de código, Segmento de datos (datos estáticos) y Segmento de Stack (datos dinámicos).
 - Identificador del usuario y grupo al que pertenece
 - Identificadores de otros procesos relacionados con él
 - Datos de los sucesos que le harán despertarse y señales pendientes.
 - Tamaño del proceso
 - Datos de su planificación
 - Terminal Original.
- **Prioridades de los procesos**

Una manera de minimizar los tiempos de respuesta del procesador para aprovechar de forma eficiente los recursos del sistema es que mientras un usuario realiza sus procesos otros usuarios pueden tener tareas en ejecución de manera simultánea.

- Dejar trabajos en baja prioridad en el sistema es una buena manera de utilizar los recursos eficientemente
- Los procesos interactivos (los que requieren entrada/salida) necesitan mayor prioridad
- El sistema ajusta las prioridades de los procesos dinámicamente, de tal forma que se puede ajustar a cambios en los procesos. Los procesos que no utilizan el procesador por un periodo aumentan su prioridad.

Nice:

El comando nice permite ejecutar un proceso en baja prioridad. Dependiendo de la versión de UNIX acepta un rango diferente de prioridades, pero los números más altos tienen prioridad más baja.

```
nice < -incremento /decremento> <comando>
```

Ejemplo: \$ nice -1vi

Estados del proceso

Durante la ejecución de un proceso su estado va cambiando, pudiendo encontrar en cualquiera de los siguientes estados:

- Estado Activo en Ejecución (Running): El proceso tiene asignado un tiempo de CPU y las instrucciones se están ejecutando.
- Estado Activo Ejecutable (Runnable): El proceso puede ser ejecutado, pero no tiene tiempo de CPU disponible.
- Estado Suspendido: Recibió una señal para detenerse (SIGSTOP). Continuará cuando reciba una señal para continuar.
- Durmiendo: Se encuentra en espera de un evento, por ejemplo, una entrada de teclado, o que
- otros procesos terminen.
- Ocioso (Idle): Fue creado pero aún no es ejecutable.
- Zombi: El proceso terminó pero su padre no ha sido notificado.

Comadnos ps

El comando ps reporta el estado de los procesos activos.

\$ ps

Si no se especifican opciones, ps mostrará información sobre los procesos que tienen los mismos user ID y controlador de terminal que la conexión desde donde se llama a ps. La información que muestra ps dependerá de las opciones especificadas, por omisión contiene el número de proceso, identificador de terminal, tiempo de ejecución acumulado y el nombre del comando que se está ejecutando

Modalidad de ps

- 1- **-a** Muestra los procesos de otros usuarios.
- 2- **-c** Muestra el nombre del comando.
- 3- **-e** Lista información sobre cada proceso en ejecución ahora.
- 4- **-f** Muestra más información sobre los procesos. Incluyendo el nombre del usuario.
- 5- **-l** Genera una lista con información detallada de los procesos.
- 6- **-u** user visualiza los procesos de dicho usuario.
- 7- **-p lista** Lista solamente los procesos cuyos números de identificación están en la lista.
- 8- **-t term** Lista los procesos asociados con la terminal. Por ejemplo, term/a, o pts/0.
- 9- **-u lista** Lista los procesos del usuario especificado(s). La lista debe estar separada por comas.
- 10- **-o formato** Muestra la información de acuerdo a un formato especificado en una lista separada por comas.

El formato se especifica usando una o más de las siguientes palabras:

user	ruid	pgid	pcpu	nice	f	psr	comm
ruser	gid	sid	pmem	class	s	tty	args
group	rgid	taskid	vsz	time	c	addr	projid
rgroup	pid	pri	rss	etime	lwp	wchan	project
uid	ppid	opri	osz	stime	nlwp	fname	pset.

Cada columna de la información que se visualiza con ps describe el estado del proceso

O - Corriendo

S - En espera de un evento sea completado.

I - Ocioso.

Z - Zombie

T - Parado por una señal del padre.

- UID El número ID del usuario.
- PID Número del proceso.
- PPID Número del proceso padre.
- PRI Prioridad del proceso.
- ADDR Dirección de memoria del proceso.
- TIME Tiempo acumulativo de ejecución.
- CMD Nombre del comando. (Nombre completo con -f)
- NI El valor de prioridad proceso.

- SZ El tamaño virtual de la imagen, calculado como el tamaño de texto+pila+datos
- RSS El número de Kilobytes del programa que está residente en la memoria actualmente
- ♣ WCHAN El número del evento del kernel por el que está esperando el proceso
- ♣ TTY El nombre del terminal de control para el proceso

Ejecución en background y foreground

Desde la línea de comando los procesos pueden crearse de dos maneras:

Primer Plano (foreground): El shell espera a que el proceso termine de ejecutarse para volver a mostrar el prompt.

Ejemplo: `$cat>tarea`

Segundo Plano (background): El proceso se ejecuta con una prioridad menor, el shell no espera a que termine y mientras tanto el usuario puede ejecutar otro comando. Para mandar una tarea (job) al background el comando se escribe finalizando con un "&". El shell notifica el PID asignado al proceso.

Ejemplo:

```
root@limesurvey ~# vi programa.c&
[2] 64877
```

Donde [2] es el número de tarea y 64877 el número PID del proceso

Jobs

A un programa corriendo en el segundo plano o background se le conoce como tarea. El comando Jobs muestra una lista de todos los procesos que un usuario está ejecutando como tarea.

```
root@limesurvey ~# jobs
[1]-  stopped                  vi programa.c
[2]+  stopped                  vi programa.c
```

La información anterior indica que la tarea1 (vi) fue detenida

Fg

El comando fg pasa una tarea del background al primer plano. Se especifica el número de tarea que se desea traer al foreground.

```
root@limesurvey ~# fg 1
```

Eliminación de procesos

El comando kill se usa para enviar señales a los procesos. Algunas de estas señales son las siguientes:

- kill -HUP <pid>: Señala al proceso con número <pid>, que vuelva a leer sus archivos de configuración
- kill -INT <pid>: Señala al proceso con número <pid>, que va a interrumpir.
- kill -TERM<pid>: Señala al proceso con numero <pid>,que debe de terminar, a diferencia de -KILL, esta opción da la oportunidad al proceso de terminar.
- kill -STOP <pid>: Señala al proceso con numero <pid>, que pare momentaneamente
- kill -CONT <pid>: Señala al proceso con numero <pid>, que continúe, este comando se utiliza para reanudar un proceso que le fue aplicado - STOP
- kill -KILL <pid>: Señala al proceso con número <pid/tarea>, que termine de inmediato.

Comando nohub

Al terminar una sesión de Unix, el sistema mata todos los procesos del usuario. Si algún proceso requiere mayor tiempo de ejecución el comando nohub permite que un proceso continúe ejecutándose aunque el usuario haya terminado su sesión.

Sintaxis:

nohup comando &

```
root@frescurvey:~# nohup find /archivo&  
[1] 64971  
root@frescurvey:~# nohup: ignoring input and appending output to 'nohup.out'
```

Signals

Las señales en Unix son la forma para avisar a un proceso que ha sucedido cierto evento y que debe ser atendido. La lista de señales a comunicar a los procesos varía muy poco entre las versiones de Unix.

Nombre	Valor	Acción	Evento
SIGHUP	1	Exit	Hangup
SIGINT	2	Exit	Interrupt
SIGQUIT	3	Core	Quit
SIGILL	4	Core	Illegal Instruction
SIGTRAP	5	Core	Trace/Breakpoint Trap
SIGABRT	6	Core	Abort
SIGEMT	7	Core	Emulation Trap
SIGFPE	8	Core	Arithmetic Exception
SIGKILL	9	Exit	Killed
SIGBUS	10	Core	Bus Error
SIGSEGV	11	Core	Segmentation Fault
SIGSYS	12	Core	Bad System Call
SIGPIPE	13	Exit	Broken Pipe
SIGALRM	14	Exit	Alarm Clock
SIGTERM	15	Exit	Terminated
SIGUSR1	16	Exit	User Signal 1
SIGUSR2	17	Exit	User Signal 2
SIGCHLD	18	Ignore	Child Status Changed
SIGPWR	19	Ignore	Power Fail/Restart
SIGWINCH	20	Ignore	Window Size Change
SIGURG	21	Ignore	Urgent Socket Condition
SIGPOLL	22	Exit	Pollable Event
SIGIO	22	Exit	input/output possible signal
SIGSTOP	23	Stop	Stopped(signal)
SIGTSTP	24	Stop	Stopped(user)
SIGCONT	25	Ignore	Continued
SIGTTIN	26	Stop	Stopped(tty input)
SIGTTOU	27	Stop	Stopped(tty output)
SIGVTALRM	28	Exit	Virtual Timer Expired
SIGPROF	29	Exit	Profiling Timer Expired
SIGXCPU	30	Core	CPU time limit exceeded
SIGXFSZ	31	Core	File size limit exceeded
SIGCKPT	33	Ignore	Checkpoint warning
SIGRESTART	34	Ignore	Restart warning
SIGRTMIN	49	Exit	POSIX1003.1b SIGRTMIN
SIGRTMAX	64	Exit	POSIX1003.1b SIGRTMAX

Tabla de Señales del sistema operativo UNIX IRIX

- 1- Genere un listado completo de todos los procesos que están en el sistema y muestre la información completa de todos los que se empezaron a ejecutar el 12 de marzo (en una sola línea).

```
$ ps aux | grep 'Mar12'
garcia07 39491 0.0 0.0 3240 708 pts/1 S+ 01:59 0:00 grep Mar12
$
```

Este comando esta coformado de 2 instrucciones unidas por un pipeline el estas son

Ps aux donde:

- **Ps:** muestra la información sobre los procesos en ejecución del sistema
- **a:** muestra procesos de todos los usuarios
- **u:** muestra los procesos con fromato detallado
- **x:** incluye procsos que no están asociados a una termina

grep “Mar12”

este toma la salida del comando que muestra los procsos y filtra esa salida para mostrar solamente las líneas que contienen “**Mar12**”

- 2- ¿Qué comando son los procesos que actualmente está ejecutando root?

```
$ ps -u root
  PID TTY          TIME CMD
    1 ?        00:00:06 systemd
    2 ?        00:00:00 kthreadd
    3 ?        00:00:00 rcu_gp
    4 ?        00:00:00 rcu_par_gp
    6 ?        00:00:00 kworker/0:0H
    8 ?        00:00:00 mm_percpu_wq
    9 ?        00:00:00 rcu_tasks_rude_
   10 ?        00:00:00 rcu_tasks_trace
   11 ?        00:00:00 ksoftirqd/0
   12 ?        00:00:12 rcu_sched
   13 ?        00:00:02 migration/0
   15 ?        00:00:00 cpuhp/0
   16 ?        00:00:00 cpuhp/1
   17 ?        00:00:02 migration/1
   18 ?        00:00:00 ksoftirqd/1
   20 ?        00:00:00 kworker/1:0H-events_highpri
   21 ?        00:00:00 cpuhp/2
   22 ?        00:00:04 migration/2
   23 ?        00:00:00 ksoftirqd/2
   25 ?        00:00:00 kworker/2:0H-events_highpri
   26 ?        00:00:00 cpuhp/3
   27 ?        00:00:06 migration/3
   28 ?        00:00:00 ksoftirqd/3
```

Aquí hacemos uso del parámetro **-u** para indicar que queremos filtrar los procesos por un usuario en específico en este caso del usuario **root** en donde el comando **ps** nos muestra todos los procesos que tiene el usuario **root**

3- Genere un listado con el número de proceso, número del proceso padre, comando en ejecución y prioridad de tres de sus compañeros

```
$ ps -eo pid,ppid,cmd,pri -u zhen02,sharai14,avalos
```

PID	PPID	CMD	PRI
1	0	/sbin/init	19
2	0	[kthreadd]	19
3	2	[rcu_gp]	39
4	2	[rcu_par_gp]	39
6	2	[kworker/0:0H]	39
8	2	[mm_percpu_wq]	39
9	2	[rcu_tasks_rude_]	19
10	2	[rcu_tasks_trace]	19
11	2	[ksoftirqd/0]	19
12	2	[rcu_sched]	19
13	2	[migration/0]	139
15	2	[cpuhp/0]	19
16	2	[cpuhp/1]	19
17	2	[migration/1]	139
18	2	[ksoftirqd/1]	19
20	2	[kworker/1:0H-events_highpr]	39

39459	611	sshd: garcia07 [priv]	19
39478	39459	sshd: garcia07@pts/1	19
39479	39478	-sh	19
39487	2	[kworker/0:1-ata_sff]	19
39488	611	sshd: zhen02 [priv]	19
39509	39488	sshd: zhen02@pts/2	19
39510	39509	-sh	19
39538	611	sshd: sharai14 [priv]	19
39557	39538	sshd: sharai14@pts/3	19
39558	39557	-sh	19
39570	2	[kworker/u8:2]	19
39571	39479	ps -eo pid,ppid,cmd,pri -u	19

Hacemos uso del parámetro **-e** que nos muestra todos los procesos en el sistema.

Tenemos que especificar que columnas serán las que queremos mostrar esas que hacen con el parámetro **-o** y las columnas a mostrar son

- **Pid:** ID del proceso
- **Ppid:** ID del proceso padre
- **Cmd:** comando ejecutado
- **Pri:** prioridad del proceso

Con el comando **-u** listamos al usuario que queremos que nos muestre sus procesos en ejecución.

4- Explique la diferencia entre las opciones de ps e,f,l y j

Aunque estos parámetros tienen la función de mostrar información sobre los procesos la diferencia radica en como lo muestra y esto es para quien quiere mostrar la información.

- **-e** : muestra todos los procesos del sistema, tanto propios como de otros usuarios
- **-f**: muestra una lista detallada en columnas de la información del proceso como **UID, PPID, C, STIME, CMD, etc**
- **-l**: muestra información mas técnica como prioridad, estados, banderas, etc.
- **-j**: muestra la información de control de trabajos, este incluye columnas como **PGID <Process Group ID>, SID <Session ID>**

5- Explique la diferencia entre las opciones de ps a y u

- **-a**: este parámetro muestra todos los procesos de todos los usuarios con una terminal activa
- **-u** : el parámetro muestra los procesos activos pero especificando de que usuario queremos ver sus procesos, como un método de monitorear que tantos recursos x usuario esta usando del sistema

6- Explique qué es lo que hace la opción de ps t y u

- **-t**: esta muestra los procesos asociados a una terminal,
- **-u**: muestra los procesos asociados a un usuario, muestra que recursos está consumiendo un usuario

Mientras que un usuario puede tener por ejemplo 2 sesiones en 2 terminales, con -u mostrara los recursos de sus 2 terminales

Pero con **-t** indicamos que terminal queremos ver los recursos que se están consumiendo. Puede ser el mismo usuario, pero son 2 terminales distintas que pueden estar ejecutando procesos diferentes o iguales.

- 7- Si tiene dos sesiones de putty abiertas con el mismo user name
- a. *¿qué procesos muestra al ejecutar ps?*

```
$ ps
      PID TTY          TIME CMD
  39479 pts/1        00:00:00 sh
  39655 pts/1        00:00:00 ps
$
```

Solo me estará mostrando los procesos asociados a la terminal en la cual ejecute **ps**

- b. *¿Qué opción de ps debería de usar para ver todos los procesos de un usuario?*

```
$ ps -u $zhen02
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
garcia07  39479  0.0  0.0   2480  1652 pts/1    Ss   01:56   0:00 -sh
garcia07  39647  0.0  0.0   2480  1616 pts/6    Ss+  02:07   0:00 -sh
garcia07  39723  0.0  0.0   6756  2912 pts/1    R+   02:09   0:00 ps -u
$
```

Para poder mostrar los procesos de un usuario si se conoce el nombre del usuario ejecutamos el comando **ps -u user_name**

- c. *¿Cómo identifico a los procesos que el usuario está ejecutando en cada terminal?*

```
$ ps -o pid, tty, cmd -u $(whoami)
      PID TT          CMD
  39478 ?          sshd: garcia07@pts/1
  39479 pts/1        -sh
  39646 ?          sshd: garcia07@pts/6
  39647 pts/6        -sh
  39779 pts/1        ps -o pid, tty, cmd -u garcia07
$
```

Para ver los procesos se ejecuta **ps -o pid, tty, cmd -u \$(whoami)** este mostrara con un formato todos los procesos activos en mis 2 terminales abiertas.

8- ¿Cuál es significado de TODAS las columnas de formato que maneja ps -o

1. pid ID del proceso.
2. ppid ID del proceso padre.
3. uid ID del usuario propietario del proceso.
4. user Nombre del usuario propietario del proceso.
5. gid ID del grupo del proceso.
6. group Nombre del grupo del proceso.
7. euid ID efectivo del usuario.
8. egid ID efectivo del grupo.
9. comm Nombre corto del comando en ejecución.
10. cmd Comando completo con argumentos.
11. args Comando con todos sus argumentos.
12. stat Estado del proceso (Ej. R running, S sleeping, Z zombie).
13. stime Hora de inicio del proceso.
14. start Fecha y hora de inicio del proceso.
15. time Tiempo total de CPU usado por el proceso.
16. %cpu Porcentaje de uso de CPU.
17. %mem Porcentaje de uso de memoria.
18. vsz Tamaño virtual del proceso en KB.
19. rss Tamaño de la memoria residente en KB.
20. tty Terminal asociada al proceso (Ej. pts/0).
21. nice Valor de prioridad (nice value).
22. pri Prioridad del proceso.
23. nlwp Número de hilos (threads) en el proceso.
24. class Clase de planificación del proceso.
25. rtprio Prioridad en tiempo real del proceso.
26. policy Política de planificación (Ej. TS time-sharing).
27. lstart Fecha y hora exacta de inicio del proceso.
28. sess ID de sesión del proceso.
29. pgid ID del grupo de procesos.
30. sid ID de sesión del usuario.
31. wchan Dirección en la que el proceso está esperando.
32. f Flags del proceso en formato hexadecimal.
33. c Contador de uso de CPU.
34. psr Núcleo de CPU donde se está ejecutando el proceso.

9- Ejecute dos comandos cualesquiera en background

```
$ vi&  
$ jobs  
[1] + Stopped (tty output)      vi  
$
```

Vi se esta ejecutando en segundo plano usando el operador **&** al final de la declaración del comando.

10-Ejecute el comando `cat >lista`, ¿Qué prioridad tiene asignada?

```
$ cat >lista &  
$ ps -o pid,pri,cmd | grep cat  
39788 19 cat  
39792 19 grep cat  
[2] + Stopped (tty input)      cat 1>lista  
$
```

Ejecutamos el comando **cat >lista&**, abre el comando **cat** en modo escritura y redirige la entrada a un archivo llamado lista y con **&** se ejecuta en segundo plano

Para poder ver la prioridad de las tareas ejecutamos **ps -o pid,pri,cmd** donde nos mostrara las columnas del identificador del proceso, la prioridad del proceso y el comando que inicio el proceso filtramos la salida **grep cat** para que nos muestre solo las tareas que tiene la palabra **cat**

11-Mate el proceso anterior.

```
$ kill -9 39788  
$ kill -9 39792  
-sh: 34: kill: No such process  
  
[2] + Killed                  cat 1>lista  
$ jobs  
[1] + Stopped (tty output)    vi
```

El proceso de matar un proceso es mediante el comando **kill** que recibe como parámetro una señal por lo general se usa **-9** el cual mata al proceso sin preguntar o sin revisar parámetros, y se usa el ID de proceso o un numero de tarea.

```
$ ps
  PID TTY          TIME CMD
 39479 pts/1        00:00:00 sh
 39785 pts/1        00:00:00 vi
 39856 pts/1        00:00:00 ps
$ kill -9 39785
$ jobs
[1] + Killed                  vi
$ ps
  PID TTY          TIME CMD
 39479 pts/1        00:00:00 sh
 39858 pts/1        00:00:00 ps
$
```

12-Vuelva a ejecutar cat>lista pero con menor prioridad

```
$ nice -n 10 cat> lista&
$ jobs
[1] + Stopped (tty input)      nice -n 10 cat 1>lista
$
```

El comando **nice** ajusta la prioridad de ejecución de un proceso con los parámetros **-n 10** se le da la prioridad ajustada por el usuario entre **mayor** sea un número menor su prioridad

13-. ¿Qué prioridad le fue asignada?

```
$ ps -o pid,pri,cmd | grep cat
39896    9 cat
39908   19 grep cat
$
```

14-Una vez más ejecute cat>lista, pero ahora en el background

```
$ jobs
[2] + Stopped (tty input)      cat 1>lista
[1] - Stopped (tty input)      nice -n 10 cat 1>lista
$
```

15-. ¿Cuál es su prioridad ahora?

```
$ ps -o pid,pri,cmd | grep cat
39896    9 cat
39942   19 cat
39951   19 grep cat
$
```

16-Verifique que el comando en background este en la lista de procesos

```
$ ps aux | grep cat
zhen02      39783  0.0  0.0   2528   560 pts/2    T   02:11   0:00 cat
garcia07    39896  0.0  0.0   2528   496 pts/1    TN  02:20   0:00 cat
garcia07    39942  0.0  0.0   2528   564 pts/1    T   02:23   0:00 cat
garcia07    39961  0.0  0.0   3240   712 pts/1    S+  02:25   0:00 grep cat
$
```

17-Verifique que el comando en background este en la lista de tareas (jobs).

```
$ jobs
[2] + Stopped (tty input)      cat 1>lista
[1] - Stopped (tty input)      nice -n 10 cat 1>lista
$
```

El comando **Jobs** se usa para ver las tareas que están en segundo plano o suspendidos

18-Pase una de las tareas al foreground use el número de tarea.

```
$ jobs
[2] + Stopped (tty input)      cat 1>lista
[1] - Stopped (tty input)      nice -n 10 cat 1>lista
$ fg
cat 1>lista
^C
$
```

Usando el comando **fg <PID/numero de tarea>** es posible pasar de modo segundo plano a foreground del sistema

19-. Pase la otra tarea al foreground, pero ahora use el número de PID.

```
$ ps
  PID TTY          TIME CMD
 39479 pts/1        00:00:00 sh
 39896 pts/1        00:00:00 cat
 40004 pts/1        00:00:00 ps
$ fg % 39896
nice -n 10 cat 1>lista
^C
$
```

20-. Envíe otro comando al background.

```
$ ps -l
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
0 S  1179    39479    39478  0   80   0  -  620  -      pts/1    00:00:00 sh
0 T  1179    40045    39479  0   80   0  - 1235  -      pts/1    00:00:00 vi
0 R  1179    40047    39479  0   80   0  - 1669  -      pts/1    00:00:00 ps
[1] + Stopped (tty output)      vi
$
```

21-Finalice este proceso

```
$ ps -l
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY      TIME CMD
0 S  1179    39479    39478  0   80   0  -  620  -      pts/1    00:00:00 sh
0 T  1179    40045    39479  0   80   0  - 1235  -      pts/1    00:00:00 vi
0 R  1179    40047    39479  0   80   0  - 1669  -      pts/1    00:00:00 ps
[1] + Stopped (tty output)      vi
$ kill -9 40045
$ jobs
[1] + Killed                    vi
$
```

22-Compruebe el funcionamiento del comando nohup.

```
$ nohup sleep 60&
$ -sh: 63: nohup: not found
^C
[1] + Done(127)                  nohup sleep 60
$ ps aux | grep sleep
garcia07  40080  0.0  0.0  3240  636 pts/1    S+   02:33   0:00 grep sleep
$
```

El comando **nohup** cuelga el proceso cuando se cierra la terminal.

Conclusiones:

Durante la practica aprendimos a conocer de lo que se compone un proceso, esto es muy importante ya que son los procesos los que dan vida, dan la forma al sistema operativo, el conocer sus atributos, conocer que usuarios esta ejecutando que proceso, es importante tanto en tema de gestionar un sistema, por ejemplo el limitar los recursos que un usuario puede acaparar en un sistema así como el como gestionar un proceso que puede que este afectando al sistema porque esta ocupando muchos recursos, con los comandos y los atributos sabemos de donde viene, hacia donde va y que esta haciendo cada uno de ellos en el sistema, el sistema operativo al final de cuenta es un conjunto de múltiples procesos ejecutándose casi al mismo tiempo, dependiendo del SO es posible o no.